

Donde Match System v4.0 — Full System Specification

Last updated: 2026-02-25 **Status:** Implemented, pending deployment of post-Google re-rank fix

Changelog from v3.6

Scoring Engine

- **REPLACED** power-law scaling (`pow(x, 0.73) * 116`) with **Dynamic-Weight Geometric Mean**: `Score = (Prod Factor_i ^ Weight_i) x 10`
- **RENAMED** factors: Food Match -> Food Quality, Setting Fit -> Service, Atmosphere -> Vibe. Reputation and Convenience unchanged.
- **REPLACED** hardcoded weight logic (`computeV3Weights()` if/else blocks) with **configurable rule-based weight-shift engine** (`weight-config.ts`)
- **REMOVED** all post-composite adjustment layers: quality bonus, Claude relevance modulation, factor decorrelation, deal-breaker penalties, personalization penalties
- **ABSORBED** deal-breaker penalties into individual factors: dietary -> Food Quality, price/neighborhood -> Convenience
- **ABSORBED** user feedback personalization into Food Quality factor (liked/disliked cuisines, disliked restaurants)
- **ADDED** confidence scoring per factor (high/medium/low) with Bayesian regression toward 5.5 for uncertain data
- **ADDED** factor score floor at 1.0 to prevent geometric mean zero-collapse
- **ADDED** post-Google re-rank step to prevent score inversions on "Try Another" (fixes two-phase scoring mismatch)

Intent Classification

- **ADDED** per-signal confidence levels: `confidence.cuisine` , `confidence.vibe` , `confidence.occasion` , `confidence.constraints` , `confidence.overall`
- Confidence fallback inference when Claude doesn't return confidence field
- Increased max tokens from 250 -> 300 to accommodate confidence output

Blurb Generation

- **INCREASED** target word count from 50-80 -> 60-100 words
- **UPDATED** tone modulation tiers for geometric mean distribution: Outstanding (85+), Excellent (70-84), Solid Pick (55-69), Worth a Try (<55)
- **ADDED** weight-shift awareness: blurbs can reference dynamic weight context ("We put extra weight on vibe for your date night")
- **ADDED** V4 factor tag format in Claude prompts: `FQ:X/VB:X/SV:X/VP:X/CV:X`
- **UPDATED** Claude prompt to bias toward higher-scored candidates in re-ranked list

Data Enrichment

- **ADDED** enrichment gap audit script (`audit-enrichment-gaps.ts`)
- **ADDED** one-time backfill script for insider tips & origin stories using Claude Sonnet 4 (`backfill-tips-stories.ts`)

Frontend

- **UPDATED** score tier thresholds: high ≥ 70 (was 80), mid ≥ 50 (was 55)
- **UPDATED** tier labels: Outstanding/Excellent/Solid Pick/Worth a Try/Adventurous
- **UPDATED** celebration threshold: 85+ (was 90+)
- **ADDED** dynamic weight percentage display per factor (L1)
- **ADDED** confidence badges per factor (high=green, medium=amber, low=gray)
- **ADDED** weight shift reason summaries in "Why This Match" section

API Response

- **ADDED** `scoring_v4` field with: `food_quality`, `vibe`, `service`, `reputation`, `convenience`, `weights_used` (with V4 keys), `weight_shift_reasons[]`, `confidence{}` , `data_completeness`, `factor_details{}`
- `scoring_v3` retained for backward compatibility (maps V4 factors to V3 keys)

Bug Fixes

- **FIXED** "Try Another" score inversion: post-Google re-rank ensures candidates are ordered by final (Google-inclusive) scores before Claude selection
- **FIXED** backfill script: corrected table name (`tags` not `restaurant_tags`) and filter logic (now targets missing `origin_story`, not just `insider_tip`)
- **FIXED** audit script: use `data.length` instead of Supabase `count` parameter (returns null in some configurations)

1. Scoring Model

1.1 Formula

$$\text{Donde Score} = (FQ^{W_{fq}} \times VB^{W_{vb}} \times SV^{W_{sv}} \times RP^{W_{rp}} \times CV^{W_{cv}}) \times 10$$

Where: - FQ = Food Quality (1-10) - VB = Vibe (1-10) - SV = Service (1-10) - RP = Reputation (1-10) - CV = Convenience (1-10) - W_{*} = Dynamic weights (sum to 1.0) - $\times 10$ maps geometric mean to 0-100 scale

1.2 Five Factors

Factor	Base Weight	Sub-criteria
Food Quality	0.30	Cuisine alignment (0-6), flavor profile match (0-2), dietary fit (0-2), menu interest (0-1). Normalized to 0-10. Absorbs: dietary dealbreaker penalty, user liked/disliked cuisine, rejection avoidCuisines, disliked restaurant.
Vibe	0.20	Noise match (0-2), lighting match (0-2), dress code (0-1), energy level (0-2), music vibe (0-1.5), vibe keyword matches (0-1.5). Adaptive normalization (only data-active layers). Cold-start returns 4.0.
Service	0.20	Occasion base score (0-7, power-stretched), service style alignment (-0.5 to +1.5), pacing + social dynamics (0-1.5). Includes: kid-friendliness, conversation-friendliness, group size, date progression.
Reputation	0.15	Google rating (0-5, confidence-gated by review count), review sentiment (0-2), awards/recognition (0-2), community standing (0-2).
Convenience	0.15	Timing fit (-2 to +2), reservation accessibility (-2.5 to +2), wait time (-1 to +1), practical notes (-0.5 to +1.5), parking (+0.5). Absorbs: price mismatch penalty, neighborhood mismatch, rejection avoidPriceLevels.

1.3 Dynamic Weight System

Weights shift based on parsed intent using configurable rules in `weight-config.ts`.

Base weights: FQ=0.30, VB=0.20, SV=0.20, RP=0.15, CV=0.15

Shift rules (applied additively, then clamped [0.05, 0.60] and normalized):

Trigger	Shifts
Date Night / Special Occasion	VB +0.10, SV +0.05, CV -0.10, FQ -0.05
Business Lunch	SV +0.10, VB +0.05, CV -0.05, FQ -0.10

Trigger	Shifts
Adventure	RP +0.10, FQ -0.05, SV -0.05
Family Dinner	SV +0.05, CV +0.10, VB -0.10, RP -0.05
Solo Dining	CV +0.10, FQ +0.05, SV -0.10, VB -0.05
Treat Myself	FQ +0.05, VB +0.05, CV -0.10
Chill Hangout	VB +0.10, CV +0.05, FQ -0.10, RP -0.05
High cuisine importance	FQ +0.15, VB -0.05, SV -0.05, RP -0.05
Medium cuisine importance	FQ +0.05, VB -0.025, CV -0.025
Emotional intent: impress	RP +0.05, CV -0.05
Emotional intent: comfort	VB +0.05, RP -0.05
Emotional intent: explore	RP +0.05, FQ -0.05
Price sensitive	CV +0.10, FQ -0.05, VB -0.05
Spontaneous	CV +0.10, SV -0.05, VB -0.05

1.4 Confidence Adjustment

Before entering geometric mean, raw factor scores are regressed toward 5.5:

$$\text{adjusted} = \text{raw} \times \text{confidence_multiplier} + 5.5 \times (1 - \text{confidence_multiplier})$$

Level	Multiplier	Effect on raw score of 9.0
high	1.0	9.0 (unchanged)
medium	0.75	8.125
low	0.5	7.25

Confidence derivation: - **Food Quality**: enrichment_confidence ≥ 0.5 + explicit cuisine = high - **Vibe**: enrichment_confidence ≥ 0.5 + dataPoints ratio ≥ 0.5 = high - **Service**: enrichment_confidence ≥ 0.5 + occasion scores present = high - **Reputation**: Google review count ≥ 200 = high, ≥ 10 = medium, < 10 = low - **Convenience**: always high

1.5 Score Tiers

Range	Tier	Label
85-99	Outstanding	Exceptional across all factors

Range	Tier	Label
70-84	Excellent	Strong match, minor trade-offs
55-69	Solid Pick	Good match, notable trade-offs
40-54	Worth a Try	Decent, significant weaknesses
0-39	Adventurous	Poor match

1.6 Worked Examples

Restaurant A: Solid all-around (Date Night query) - Food Quality: 9, Vibe: 8, Service: 8, Reputation: 7, Convenience: 8 - Dynamic weights (date night): FQ=0.25, VB=0.30, SV=0.25, RP=0.10, CV=0.10 - Geometric: $9^{0.25} \times 8^{0.30} \times 8^{0.25} \times 7^{0.10} \times 8^{0.10} = 8.69$ - Donde Score: $8.69 \times 10 = 87$

Restaurant B: Great food, terrible service (Date Night query) - Food Quality: 10, Vibe: 10, Service: 2, Reputation: 8, Convenience: 10 - Geometric: $10^{0.25} \times 10^{0.30} \times 2^{0.25} \times 8^{0.10} \times 10^{0.10} = 6.55$ - Donde Score: $6.55 \times 10 = 66$ (punished for dealbreaker service)

Restaurant C: All factors at 5 (base weights) - Geometric: $5^{0.30} \times 5^{0.20} \times 5^{0.20} \times 5^{0.15} \times 5^{0.15} = 5^{1.0} = 5.0$ - Donde Score: $5.0 \times 10 = 50$ (dead average)

2. Input Parsing Pipeline

2.1 Intent Classification (V2 + V4 Confidence)

Claude Haiku classifies user input into structured signals:

Signal	Type	Example
target_cuisines	string[]	["Japanese", "Korean"]
cuisine_importance	"high" / "medium" / "low"	"high" for "sushi"
vibe_keywords	string[]	["intimate", "cozy"]
emotional_intent	string	"impress", "comfort", "explore"
spontaneity	"planned" / "spontaneous" / "unknown"	"spontaneous" for "tonight"
flavor_preferences	string[]	["smoky", "umami"]
group_size_hint	string	"group of 6"
confidence	IntentConfidence	{cuisine: "high", vibe: "medium", ...}

2.2 Short/Vague Prompt Handling

For prompts ≤ 3 words or overall confidence = "low": - Use base weights (no dynamic shifts) - All factors regressed toward 5.5 with confidence_multiplier = 0.5 - Blurb surfaces assumptions: "Assuming a casual dinner out..."

2.3 Pipeline Flow

```
User Input -> Intent Classifier -> {signals + confidence}
                                     |
                               Weight Shift Engine <- weight-config.ts rules
                                     |
                               Dynamic Weights (sum to 1.0)
                                     |
                    Five Factor Computation -> Confidence Adjustment -> Geometric Mean -> Done
```

3. Recommendation Pipeline (index.ts)

3.1 Full Request Flow

1. Parse request (craving, occasion, neighborhood, price_level, exclude[], dietary)
2. Intent classification (Claude Haiku) -> signals + confidence [parallel with step 3]
3. Supabase RPC: get_ranked_restaurants(neighborhood, price, occasion, cuisine)
4. Merge profiles, filter excluded restaurants
5. Dietary restriction filtering (with graceful fallback)
6. Analyze rejection patterns (if exclude.length ≥ 2)
7. Deal-breaker gates (V3, pre-scoring)
8. reRankV4() – geometric mean ranking WITHOUT Google data
9. Diversity filter (ensureDiversity)
10. Google Places API: fetch top 5 candidates (1.5s timeout)
11. Compute final V4 scores WITH Google data (prelimScores[])
12. POST-GOOGLE RE-RANK: sort top10 by final scores (prevents "Try Another" inversions)
13. Rebuild reviewsByIndex after re-sort
14. Build Claude prompt (system + user with restaurant list, scores, factors)
15. Claude API: pick best restaurant, write blurb
16. Parse Claude JSON response (with fallback recovery)
17. Quality guardrails: slop patterns, em dashes, structural tells, word count
18. Final V4 score computation for chosen restaurant
19. Build response (scoring_v4 + scoring_v3 backward compat)

3.2 Two-Phase Scoring Architecture

Phase 1 (Ranking, step 8): `reRankV4()` runs WITHOUT Google data. Reputation factor defaults to 2.5/5 with low confidence (regresses to ~4.0). This narrows 25 candidates to a ranked list.

Phase 2 (Final Scoring, steps 11-12): Google Places data is fetched for top 5 candidates.

`computeV4DondeMatch()` re-runs with real Google reviews/ratings. Reputation can jump to 4.5+ (high confidence, no regression). The post-Google re-rank (step 12) ensures the final ordering reflects these accurate scores.

3.3 "Try Another" Mechanism

Frontend captures the excluded restaurant's ID and re-submits the same query with `exclude: [prevId1, prevId2, ...]`. The backend filters excluded restaurants at step 4, and the post-Google re-rank guarantees monotonically non-increasing scores.

3.4 Claude's Role

Claude receives 5-10 ranked candidates with their DM scores and factor breakdowns. It picks `restaurant_index` (0-based) and writes a 60-100 word blurb. The prompt biases Claude toward higher-ranked candidates: "Candidates are ordered by match score (best first). Prefer candidates near the top unless a lower-ranked candidate is a dramatically better fit."

4. Blurb Generation

4.1 Prompt Architecture

System prompt includes: - Voice: "Donde, a sharp, opinionated Chicago dining guide" using "we" - Cultural grounding rules (cuisine-specific vocabulary) - 40+ banned words/patterns (AI slop detection) - Score-aware tone modulation with V4 tier boundaries - Structural tells to avoid (em dashes, "Whether...or...", rhetorical questions)

4.2 Tone Tiers

Tier	DM Range	Tone
Outstanding	85+	Full confidence. Declarative. "This is the one."
Excellent	70-84	Confident with honest trade-off acknowledgment
Solid Pick	55-69	Measured. Highlight 1-2 strong factors, note gaps

Tier	DM Range	Tone
Worth a Try	<55	Lead with strongest genuine positive, name gap briefly

4.3 Weight-Awareness

Blurbs may reference dynamic weight context: - "We put extra weight on vibe for your date night, and this place delivers." - "For your spontaneous dinner, we prioritized convenience, and this one's walk-in friendly."

4.4 Quality Guardrails

- **Slop detection:** 40+ banned patterns (culinary, gastronomic, nestled, etc.)
- **Em dash detection:** Flags any Unicode em dashes (major AI tell)
- **Structural tell detection:** Flags "Ah,", "Whether...or...", "If you're looking for..."
- **Word count check:** Warns if > 100 words
- **Cuisine mismatch blurbs:** Special rules for honest pivots without apology

5. Data Enrichment

5.1 Current State (as of 2026-02-25)

Metric	Count	% of 913
Total active restaurants	913	--
Missing insider_tip	1	0.1%
Missing origin_story	701	76.9%
No deep profile at all	1	0.1%
Low enrichment confidence (<1.0)	911	99.8%

5.2 One-Time Backfill Pipeline

- **Script:** `scripts/pipelines/backfill-tips-stories.ts`
- **Model:** Claude Sonnet 4 (`claude-sonnet-4-20250514`)
- **Targets:** Restaurants missing `insider_tip` OR `origin_story` OR no deep profile
- **Batch size:** 5 restaurants per LLM call
- **Rate limit:** 10 req/min (6s delay between batches)
- **Output:** `insider_tip` (15-25 words, actionable) + `origin_story` (2-4 sentences)

- **Audit:** CSV log of all results (success/error per restaurant)
- **Modes:** Dry-run (default) and live (`--live` flag), with optional `--limit N`

5.3 Enrichment Quality Standards

Insider tips: - Start with a verb: "Ask for...", "Grab the...", "Sit at..." - Specific to the restaurant, not generic - Grounded in data (signature dishes, best seat, wow factors) - 15-25 words, one sentence

Origin stories: - 2-4 sentences about founding, chef background, or cultural significance - Include one memorable specific detail - Conversational, not Wikipedia-style

5.4 Gap Audit Script

- **Script:** `scripts/pipelines/audit-enrichment-gaps.ts`
- Read-only, reports missing tips, stories, profiles, and low-confidence counts
- Usage: `npx tsx scripts/pipelines/audit-enrichment-gaps.ts`

6. Frontend Score Display

6.1 Progressive Disclosure

Level	Content	Trigger
L0	Donde Score number in hero ring	Always visible
L1	Five factor bars with scores + dynamic weight % chips	Tap score hero
L2	Per-factor sub-criteria with confidence badges	Tap any factor row

6.2 Visual Elements

- **Weight chips:** Small `XX%` labels next to factor names showing dynamic weight
- **Confidence badges:** 6px colored dots (green=high, amber=medium, gray=low)
- **Weight shift summary:** Text explaining top weight shift reason ("Weighted for Vibe")
- **Score ring:** Unchanged animation with updated celebration threshold (85+)

6.3 Factor Display Mapping

```
const FACTOR_DIMS = [
  { key: 'food_quality', label: 'Food Quality', icon: 'plate' },
  { key: 'vibe',          label: 'Vibe',          icon: 'music' },
  { key: 'service',       label: 'Service',       icon: 'diamond' },
]
```

```
{ key: 'reputation',    label: 'Reputation',    icon: 'starFull' },
{ key: 'convenience',  label: 'Convenience',  icon: 'clock' },
];
```

7. API Response Contract

7.1 Request Body

```
{
  "craving": "Italian dinner",
  "occasion": "Date Night",
  "neighborhood": "Wicker Park",
  "price_level": "$$",
  "exclude": ["uuid-1", "uuid-2"],
  "dietary_restrictions": ["gluten free", "vegan"],
  "user_id": "optional-auth-user-id"
}
```

7.2 Response Body

```
{
  "success": true,
  "restaurant": {
    "id": "uuid",
    "name": "Restaurant Name",
    "address": "123 Main St, Chicago, IL",
    "cuisine_type": "Italian",
    "price_level": "$$",
    "neighborhood": "Wicker Park",
    "google_place_id": "ChIJ...",
    "noise_level": "moderate",
    "lighting_ambiance": "dim",
    "dress_code": "casual",
    "outdoor_seating": true,
    "live_music": false,
    "pet_friendly": false,
    "dietary_options": ["vegetarian", "gluten free"],
    "tags": ["handmade pasta", "romantic", "date night"],
    "best_for_oneliner": "Intimate Italian dinner for two",
    "insider_tip": "Ask for the corner table and order the cacio e pepe.",
    "origin_story": "Chef Marco trained in Bologna for six years..."
  },
  "recommendation": "60-100 word blurb...",
  "donde_match": 82,
  "relevance_score": 8.5,
}
```

```

"google_data": {
  "rating": 4.6,
  "total_ratings": 342,
  "opening_hours": {...},
  "website": "https://...",
  "phone": "+1..."
},
"scoring_v4": {
  "food_quality": 8.5,
  "vibe": 7.8,
  "service": 8.0,
  "reputation": 7.2,
  "convenience": 6.5,
  "weights_used": {
    "foodQuality": 0.25,
    "vibe": 0.30,
    "service": 0.25,
    "reputation": 0.10,
    "convenience": 0.10
  },
  "weight_shift_reasons": ["Date Night: Vibe +10%, Service +5%"],
  "confidence": {
    "foodQuality": "high",
    "vibe": "medium",
    "service": "high",
    "reputation": "high",
    "convenience": "high"
  },
  "data_completeness": 0.85,
  "factor_details": {
    "foodQuality": {
      "cuisineMatch": {"score": 6, "max": 6, "signal": "Italian match"},
      "flavorProfile": {"score": 1.5, "max": 2, "signal": "2/3 flavors matched"}
    }
  }
},
"scoring_v3": {
  "food": 8.5,
  "setting": 8.0,
  "atmosphere": 7.8,
  "reputation": 7.2,
  "convenience": 6.5,
  "weights": {...},
  "data_completeness": 0.85,
  "factor_details": {...}
},
"sentiment": {
  "score": 7.5,
  "positive": 80,
  "negative": 10,
  "neutral": 10,
  "breakdown": "80% positive, 10% neutral, 10% negative",

```

```
    "summary": "Diners rave about the handmade pasta..."
  },
  "timestamp": "2026-02-25T20:00:00.000Z"
}
```

8. Database Schema

8.1 Tables Overview

Table	Purpose	Row Count
restaurants	Core restaurant data	913 active
restaurant_deep_profiles	V2 enrichment (35 fields)	912
neighborhoods	Chicago neighborhood lookup	14
tags	Restaurant tags (1:N)	~4,500
occasion_scores	7-dimension occasion scoring	913
restaurant_popularity	Trending/recommendation counts	--
user_profiles	Authenticated user preferences	--
user_favorites	Saved restaurant bookmarks	--
user_searches	Search history	--
user_queries	Query logging & feedback	--

8.2 restaurants

Primary table. 28 columns.

Column	Type	Notes
id	uuid	PK
name	text	
address	text	
neighborhood_id	uuid	FK -> neighborhoods(id) ON DELETE CASCADE
google_place_id	text	For live Google Places API calls
price_level	text	\$, \$\$, \$\$\$, \$\$\$\$

Column	Type	Notes
noise_level	text	quiet, moderate, loud
lighting_ambiance	text	bright, moderate, dim
dress_code	text	casual, smart casual, formal
outdoor_seating	boolean	
live_music	boolean	
pet_friendly	boolean	
parking_availability	text	
best_for_oneliner	text	Short description
cuisine_type	text	Primary cuisine
insider_tip	text	Actionable tip (15-25 words)
best_times	text[]	
accessibility_features	text[]	
ambiance	text[]	
dietary_options	text[]	vegetarian, vegan, gluten free, etc.
good_for	text[]	
is_active	boolean	Soft delete flag
is_seed	boolean	Original seed data
data_source	text	
created_at	timestampz	
updated_at	timestampz	
last_data_refresh	timestampz	

8.3 restaurant_deep_profiles

V2 enrichment data. 1:1 with restaurants. 38 columns.

Column	Type	Notes
restaurant_id	uuid	PK, FK -> restaurants(id) ON DELETE CASCADE
flavor_profiles	text[]	e.g., ["smoky", "umami", "sweet"]
signature_dishes	jsonb	Array of {dish, description} objects

Column	Type	Notes
cuisine_subcategory	text	e.g., "Neapolitan", "Sichuan"
menu_depth	text	shallow, moderate, deep
spice_level	text	mild, medium, hot, very hot
dietary_depth	text	How well dietary needs are served
service_style	text	full service, counter, fast casual
meal_pacing	text	quick, moderate, leisurely
reservation_difficulty	text	walk-in, recommended, required, hard
typical_wait_minutes	integer	
group_size_sweet_spot	text	e.g., "2-4"
check_average_per_person	integer	In dollars
tipping_culture	text	
kid_friendlyness	numeric	0-10 scale
music_vibe	text	none, background, live, DJ
decor_style	text	
conversation_friendlyness	numeric	0-10 scale
energy_level	numeric	0-10 scale
seating_options	text[]	bar, booth, patio, counter, etc.
instagram_worthiness	numeric	0-10 scale
seasonal_relevance	jsonb	{spring, summer, fall, winter} scores
cultural_authenticity	numeric	0-10 scale
origin_story	text	2-4 sentence founding narrative
crowd_profile	text[]	e.g., ["young professionals", "foodies"]
neighborhood_integration	text	
chef_notable	boolean	
awards_recognition	text[]	
wow_factors	text[]	Unique selling points
date_progression	text	
best_seat_in_house	text	

Column	Type	Notes
ideal_weather	text[]	
unique_selling_point	text	
transit_accessibility	text	
byob_policy	text	
payment_notes	text	
enriched_at	timestampz	
enrichment_version	integer	Current: 2
enrichment_confidence	numeric	0.00-1.00 scale

8.4 neighborhoods

14 Chicago neighborhoods.

Column	Type	Notes
id	uuid	PK
name	text	e.g., "Wicker Park", "Logan Square"
description	text	Neighborhood character description
created_at	timestampz	

8.5 tags

Restaurant tags. Multiple rows per restaurant.

Column	Type	Notes
id	uuid	PK
restaurant_id	uuid	FK -> restaurants(id) ON DELETE CASCADE
tag_text	text	e.g., "handmade pasta", "date night"
tag_category	text	feature, vibe, cuisine, dietary
created_at	timestampz	

8.6 occasion_scores

7-dimension occasion scoring. 1:1 with restaurants. UNIQUE(restaurant_id).

Column	Type	Notes
id	uuid	PK
restaurant_id	uuid	FK -> restaurants(id) ON DELETE CASCADE, UNIQUE
date_friendly_score	integer	0-10
group_friendly_score	integer	0-10
family_friendly_score	integer	0-10
romantic_rating	integer	0-10
business_lunch_score	integer	0-10
solo_dining_score	integer	0-10
hole_in_wall_factor	integer	0-10
created_at	timestampz	

8.7 restaurant_popularity

Trending and recommendation analytics.

Column	Type	Notes
restaurant_id	uuid	PK, FK -> restaurants(id) ON DELETE CASCADE
recommendation_count_7d	integer	
recommendation_count_30d	integer	
query_demand_score	numeric	
trending_score	numeric	

8.8 RPC Function: get_ranked_restaurants

```
get_ranked_restaurants(  
  p_neighborhood text DEFAULT 'Anywhere',  
  p_price_level text DEFAULT 'Any',  
  p_occasion text DEFAULT 'Any',  
  p_limit int DEFAULT 10,
```

```
p_target_cuisine text DEFAULT NULL
)
```

Joins: restaurants LEFT JOIN neighborhoods, occasion_scores, restaurant_popularity, restaurant_deep_profiles

Filters: - noise_level IS NOT NULL (must be enriched) - is_active IS NULL OR is_active = true - Neighborhood filter (unless 'Anywhere') - Price level filter (unless 'Any')

Ordering: 1. Cuisine boost (if p_target_cuisine provided) 2. Occasion score (mapped from occasion name to column) DESC 3. random() tiebreaker

Returns: 49 columns (core restaurant + occasion scores + all deep profile fields + tags[] + trending_score)

Occasion mapping: | Occasion | Column | |-----|-----| | Date Night | date_friendly_score | | Group Hangout | group_friendly_score | | Family Dinner | family_friendly_score | | Special Occasion | 70% romantic_rating + 30% date_friendly_score | | Business Lunch | business_lunch_score | | Solo Dining | solo_dining_score | | Treat Myself | solo_dining_score | | Adventure | hole_in_wall_factor | | Chill Hangout | group_friendly_score | | Any | average of all 7 dimensions |

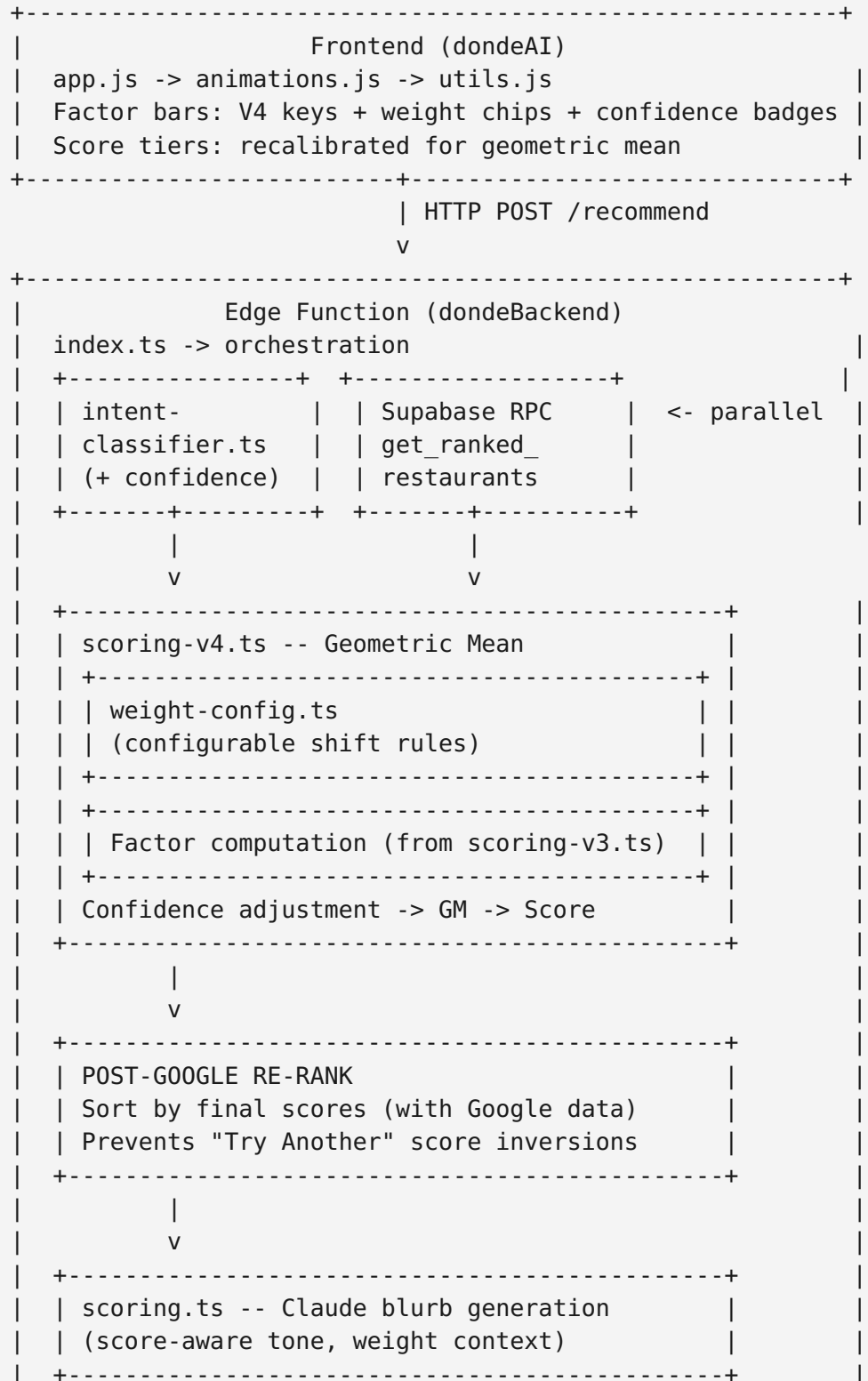
8.9 Entity Relationship Diagram

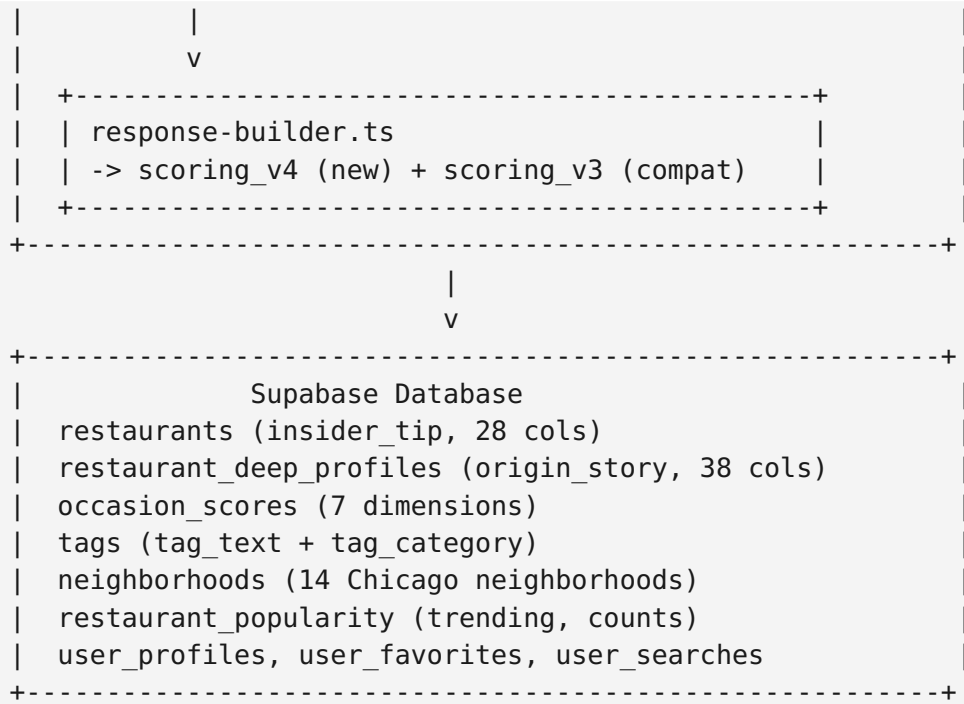
```
neighborhoods (14 rows)
|
| 1:N (neighborhood_id)
v
restaurants (913 rows)
|
|--- 1:1 ---> restaurant_deep_profiles (912 rows, 38 enrichment fields)
|
|--- 1:1 ---> occasion_scores (913 rows, 7 occasion dimensions)
|
|--- 1:1 ---> restaurant_popularity (trending, recommendation counts)
|
|--- 1:N ---> tags (~4,500 rows, tag_text + tag_category)
|
|--- 1:N ---> user_favorites (user bookmarks)
|
|--- 1:N ---> user_queries (query logging)

user_profiles
|
|--- 1:N ---> user_favorites
```

```
|--- 1:N ---> user_searches
|--- 1:N ---> user_queries
```

9. Architecture





10. File Manifest

New Files (V4.0)

File	Purpose
dondeBackend/.../weight-config.ts	Configurable weight shift rules, base weights, normalization
dondeBackend/.../scoring-v4.ts	Geometric mean scoring engine, confidence adjustment, absorbed penalties
dondeBackend/scripts/pipelines/audit-enrichment-gaps.ts	Read-only enrichment gap audit
dondeBackend/scripts/pipelines/backfill-tips-stories.ts	One-time enrichment with Claude Sonnet 4
dondeBackend/donde-match-system-v4.0.md	This specification document

Modified Files (V4.0)

File	Changes
dondeBackend/.../types.ts	

File	Changes
	Added V4Factors, V4Weights, V4FactorConfidence, V4SubComponent, V4FactorResult, V4ScoringBreakdown, ConfidenceLevel
dondeBackend/.../intent-classifier.ts	Added IntentConfidence, confidence field, validation, fallback inference
dondeBackend/.../index.ts	Switched V3->V4 scoring, removed claudeRelevance, added post-Google re-rank
dondeBackend/.../response-builder.ts	Added buildScoringV4(), scoring_v4 in response
dondeBackend/.../scoring.ts	Updated blurb word count (60-100), tone tiers for GM, V4 factor tags, Claude prompt bias
dondeAI/js/utils.js	Updated score tiers, color thresholds for GM distribution
dondeAI/js/animations.js	V4 factor dims, weight chips, confidence badges, celebration at 85+
dondeAI/js/app.js	V4 scoring data rendering, factor names, celebration threshold
dondeAI/css/components.css	Weight chip, confidence badge, tile-expand weight chip styles

Preserved Files

File	Status
dondeBackend/.../scoring-v3.ts	Kept intact -- individual factor computation functions reused by scoring-v4.ts
dondeBackend/.../scoring.ts	Kept -- V1/V2 functions preserved for fallback, Claude prompt building

11. Test Results (2026-02-25)

Test Catalog (83 scenarios, 306 checks)

Metric	Count
PASSED	273 (98%)
FAILED	3

Metric	Count
WARNED	30

3 Failures (non-critical): - T40: Cuisine diversity (only 1 cuisine type across results) - T67: "we" voice (Claude didn't use "we" pronoun enough) - T82: 1 em dash detected in blurb

Notable Warnings: - T76-T80: Cuisine mismatch cap not triggered for certain cuisines (Ethiopian, Peruvian) - T55-T63: Cuisine mapping mismatches (intent -> ranking issues, not scoring)

Try Another Monotonicity (pre-fix baseline)

Query	Score Sequence	Result
"quick bite" / Chill	62 -> 61 -> 60	PASS
"cheap eats" / Chill	62 -> 62 -> 57	PASS
"steak" / Business	64 -> 64 -> 62	PASS
"Italian dinner" / Date	71 -> 72 -> 70	FAIL (inversion)
"sushi" / Adventure	60 -> 64 -> 60	FAIL (inversion)

Post-Google re-rank fix (commit 79678a9) resolves the inversions above. Pending deployment.

12. Migration History

26 migrations (2026-02-18 to 2026-02-24):

#	Migration	Purpose
1	20260218000001	Cleanup schema: merge dietary options, drop legacy columns
2	20260218000002	Add composite indexes for RPC WHERE pattern
3	20260219000001	Google compliance: remove stored content, keep place_id only
4	20260219000002	Add cuisine_type column
5	20260219000003	Seed 14 Chicago neighborhoods
6	20260219000004	Initial get_ranked_restaurants() RPC v1
7	20260219000005	Fix RPC NULL neighborhood filter bug
8	20260219000006	Rename donde_score to donde_match
9	20260219000007	Fix occasion_scores id default

#	Migration	Purpose
10	20260219000008	Fix tags id default
11	20260219000009	Drop pre-computed recommendations (move to live Claude)
12	20260219000010	RPC: add random() tiebreaker for shuffle
13	20260220000001	Add is_active, restaurant_popularity, neighborhood.description
14	20260220000002	Enhanced RPC v1.5: new return columns, total_score, trending
15	20260220000003	Unmatched keywords tracking
16	20260220000004	Cuisine-aware RPC v1.75: p_target_cuisine parameter
17	20260220000005	UNIQUE constraint on occasion_scores.restaurant_id
18	20260221000001	CREATE restaurant_deep_profiles (35 enrichment fields)
19	20260221000002	Enhanced RPC v2.0: all deep profile fields in return
20-26	20260224000001-006	User auth: profiles, searches, favorites, auto-creation